

TOPIC:

STRESS DETECTION

INDEX

S.NO.	TOPIC
1	CERTIFICATE
2	ACKNOWLEDGEMENTS
3	INTRODUCTION
4	OPERATION
5	WORK FLOW
6	FUNCTIONS USED
7	PYTHON CODES
8	OUTPUTS
9	FUTURE SCOPE
10	BIBLIOGRAPHY

CERTIFICATE

This is to certify that Pari Yadav has successfully prepared the report on the project entitled “Stress Management” under the guidance of Ms. Sony Devatwal. The report is the result of her efforts & endeavours. She has successfully completed the research on the above-mentioned project during the year 2025-26.

ACKNOWLEDGEMENT

I would like to express our sincere gratitude to our Computer Science teacher Ms. Sony Devatwal for her valuable guidance, constant encouragement, and support throughout the completion of this project on Stress Management. Her suggestions and feedback helped us understand the topic better and complete the project successfully.

I am also thankful to Ms. Suruchi Gandhi for her motivation and cooperation, which inspired us to put in our best efforts while working on this project.

I would like to express our heartfelt thanks to our parents for their continuous support, encouragement, and understanding, without which this project wouldn't have been possible.

Lastly, I am grateful to all those who directly or indirectly helped us in successfully completing this project.

]

,

INTRODUCTION

In today's fast-paced and technology-driven world, stress has become a common problem affecting people of all age groups, especially students. Academic pressure, increased screen time, lack of sleep, and unhealthy daily routines are some of the major causes of stress. If stress is not identified and managed at the right time, it can negatively affect both physical and mental health.

The Stress Management System is a computer-based project developed using Python programming language and MySQL database. The main objective of this project is to record, analyze, and manage stress levels of users based on various activities such as sleep hours, heart rate, and screen time. The system stores user details, activity data, and stress analysis records in a structured database and provides stress levels along with suitable suggestions.

This project helps in understanding how data can be stored, retrieved, and analyzed efficiently using database connectivity. It also demonstrates the practical use of Python functions, menus, and SQL queries to perform operations like adding records, searching data, displaying reports, and managing stress analysis information.

Overall, this project highlights the importance of stress management and shows how technology can be used effectively to analyze stress and provide helpful recommendations for a healthier lifestyle.

OPERATIONS

The stress management system provides various operations to efficiently store, manage, and analyse user stress data using Python and MySQL. The main operations of the system are described below:

1. User Record Operations

This module manages personal details of users.

- Add User Record : Allows entry of user details such as UserID, Name, Age, Gender and Occupation.
- Display User Record : Displays all the user records stored in database
- Search User Record : Searches user information using user ID or user name
- Update User Record: Updates user details like age, gender and occupation.
- Delete User Record: Deletes a user record from the databases after confirmation

2. Activity Data Operations

This operation manages daily activity information of users

- Add Activity Data : Stores activity details such as sleep hours, heart rate, and screen time
- Display Activity Data : Displays all activity records present in the database
- Search Activity Data : Searches activity records using activity ID
- Update Activity Data : Updates activity details like sleep hours
- Delete Activity Data : Deletes activity data from the database.

3. Stress Analysis Operations

This operation analyses stress levels based on activity data

- Add Stress Analysis Record : Calculates stress level based on sleep hours, heart rate and screen time
- Display Stress Analysis Report : Displays complete stress analysis reports by combining users, activity and stress data
- Search Stress Data : Allows searching stress records using activity ID, user ID, or analyst ID

4. Menu-Driven Interface

The project uses a menu driven programs that allow user to easily perform all user, activity, and stress analysis operations.

WORK FLOW

The project starts with connecting Python to the MySQL database named stress_detection.

1. User details such as user ID, name, age, gender, and occupation are entered and stored in the userrecord table.
2. activity details of the user including sleep hours, heart rate, and screen time are entered and stored in the activity_data table.
3. The system then retrieves activity data from the database and analyzes it using predefined conditions.
4. Based on sleep hours, heart rate, and screen time, the stress level is calculated as Low, Medium, or High.
5. The analyzed stress level and suitable suggestions are stored in the stress_analyst table.
6. The system generates a stress analysis report by combining user data, activity data, and stress analysis results.
7. The user can search stress analysis reports using user ID, activity ID, or analyst ID.
8. All operations are performed through a menu-driven interface, making the system easy to use.
9. The program ends after all required operations are completed.

FUNCTIONS USED

FUNCTION NAME	USED FOR
add_user_data()	Inserts new user details (user id, name, age, gender, occupation) into the userrecord table.
display_userrecord_data()	Displays all user records stored in the userrecord table.
search_userrecord()	Searches user details based on user ID or user name.
update_userrecord()	Updates user information such as age, gender, and occupation.
user_delete()	Deletes a user record from the userrecord table after confirmation.
add_activity_data()	Inserts activity details such as sleep hours, heart rate, and screen time into the activity_data table.
display_activity_data()	Displays all activity records stored in the activity_data table
search_activity_data()	Searches activity records using activity ID.
update_activity_data()	Updates activity details like sleep hours.

delete_activity_data()	Deletes activity records from the activity_data table.
add_record_stress_analysis()	Analyzes stress level based on activity data and stores stress level and suggestion in stress_analyst table.
display_record_stress_analysis()	Displays complete stress analysis report by joining user, activity, and stress tables
search_by_activity_id()	Searches stress analysis records using activity ID
search_by_user_id()	Searches stress analysis records using user ID.
search_by_analyst_id()	Searches stress analysis records using analyst ID.
create_userrecord()	Creates the userrecord table in the database.
create_activity_data()	Creates the activity_data table in the database.
create_stress_analyst()	Creates the stress_analyst table in the database

PYTHON CODE

1. create_table.py

```
import mysql.connector as ms

conn=ms.connect(host="localhost", user="root", password="sony",
database="stress_detection")

curl=conn.cursor()

if conn.is_connected():
    print("connected")
else:
    print("not connected")


def create_userrecord():
    query_user="""create table userrecord
    (user_id varchar(10) NOT NULL PRIMARY KEY,
    uname varchar(50),
    uage int,
    ugender varchar(30),
    uoccupation varchar(50)
    )"""
    curl.execute(query_user)
    print("Table user created")


def create_activity_data():
    query_activity_data="""create table activity_data
    (activity_id varchar(10) NOT NULL PRIMARY KEY,
    sleep_hours int,
    heart_rate varchar(10),
    screen_time int
```

```

)"""
curl.execute(query_activity_data)
print("table activity_data created")

def create_stress_analyst():
    query_stress_analyst="""create table stress_analyst
(analyst_id varchar(10) NOT NULL PRIMARY KEY,
user_id varchar(10),
activity_id varchar(10),
stress_level varchar(30),
suggestion varchar(100)
)"""
    curl.execute(query_stress_analyst)
    print("table stress_analyst created")

create_stress_analyst()
create_activity_data()
create_userrecord()

```

2. Userrecord.py

```

import mysql.connector as ms

cnx=ms.connect(host="localhost", user="root", passwd="1234", db="stress_detection")

def add_user_data():
    cursor=cnx.cursor()
    myuser_id=input("Enter user id")
    myuname=input("Enter user name")
    myuage=int(input("Enter age"))

```

```
myugender=input("Enter gender")
myuoccupation=input("Enter the occupation")
query="insert into userrecord(user_id, uname, uage, ugender, uoccupation)
VALUES(%s,%s, %s, %s,%s)"
values=(myuser_id, myuname, myuage, myugender, myuoccupation)

cursor.execute(query,values)
cnx.commit()
cursor.close()
print("RECORD INSERTED")
```

```
def display_userrecord_data():
    cursor=cnx.cursor()
    query=("select * from userrecord")
    cursor.execute(query)
    record=cursor.fetchall()
    for row in record:
        print("User id=", row[0])
        print("User name=", row[1])
        print("User age=", row[2])
        print("User gender=", row[3])
        print("User occupation=", row[4])
        print("Data Displayed")
        print()
    cursor.close()
```

```
def search_userrecord_by_name(myuname):
    cursor=cnx.cursor()
```

```
query="select * from userrecord where uname=%s"
cursor.execute(query,(myuname,))
record=cursor.fetchall()
for row in record:
    print("Records of the data is as follow")
    print("User id=", row[0])
    print("User name=", row[1])
    print("User age=", row[2])
    print("User gender=", row[3])
    print("User occupation=", row[4])
    print("Data Displayed")
cursor.close()
```

```
def search_userrecord_by_id(unno):
    cursor=cnx.cursor()
    query="select * from userrecord where user_id= %s"
    cursor.execute(query,(unno,))
    record=cursor.fetchall()
    for row in record:
        print("Records of the data is as follow")
        print("User id=", row[0])
        print("User name=", row[1])
        print("User age=", row[2])
        print("User gender=", row[3])
        print("User occupation=", row[4])
        print("Data Displayed")
    cursor.close()
```

```

def update_userrecord(myuname):
    cursor=cnx.cursor(buffered=True)
    cursor.execute("select * from userrecord where uname='%s' ")
    print("Enter the new data")
    age=int(input("Enter user age"))
    gender=input("Enter user gender")
    occu=input("Enter user occupation")
    cursor.execute("update userrecord set\
        uage= %s, ugender= '%s', uoccupation='%s'\
        where uname='%s'" %(age, gender, occu,myuname))
    cnx.commit()

    print("Record(s) updated successfully")
    cursor.close()

```

```

def user_delete(user_id):
    cursor=cnx.cursor()
    cursor.execute("select * from userrecord where user_id=%s" , (user_id,))
    record=cursor.fetchall()
    print("Displaying record for deletion")
    print(record)
    confirm=input("Do you want to delete the record")
    if confirm=="y":
        cursor.execute("delete from userrecord where user_id='%s' " %user_id)
        cnx.commit()
        print(cursor.rowcount, "records deleted succesfully")
    else:

```

```
print("delete operation cancelled")

cursor.close()
```

3. **activityclass.py**

```
import mysql.connector as ms
```

```
cnx = ms.connect(
    host="localhost",
    user="root",
    passwd="1234",
    database="stress_detection"
)
```

```
def add_activity_data():
    cursor = cnx.cursor()
    activity_id = input("Enter activity id: ")
    sleep_hours = int(input("Enter sleep hours: "))
    heart_rate = input("Enter heart rate: ")
    screen_time = int(input("Enter screen time: "))
```

```
query = """
```

```
INSERT INTO activity_data
```

```
(activity_id, sleep_hours, heart_rate, screen_time)
```

```
VALUES (%s, %s, %s, %s)
```

```
"""
```

```
values = (activity_id, sleep_hours, heart_rate, screen_time)
```

```
cursor.execute(query, values)
```



```
cnx.commit()

cursor.close()

print("RECORD INSERTED IN ACTIVITY TABLE")
```

```
def display_activity_data():

    cursor = cnx.cursor()

    print("Displaying data from activity table")

    cursor.execute("SELECT * FROM activity_data")

    record=cursor.fetchall()

    for row in record:

        print("records of the data is as follow")

        print("Activity id=", row[0])

        print("Sleep hours=", row[1])

        print("Heart rate=", row[2])

        print("Screen time=", row[3])


    cursor.close()
```

```
def search_activity_data(aid):

    cursor = cnx.cursor()

    cursor.execute("SELECT * FROM activity_data WHERE activity_id = %s", (aid,))

    records = cursor.fetchall()

    print("ACTIVITY DATA REPORT")

    for row in records:

        print("ActivityID:", row[0])
```

```
print("Sleep Hours:", row[1])  
print("Heart Rate:", row[2])  
print("Screen Time:", row[3])  
cursor.close()
```

```
def update_activity_data(aid):  
    cursor = cnx.cursor()  
    new_sleep = int(input("Enter new sleep hours: "))  
  
    query = """  
    UPDATE activity_data  
    SET sleep_hours = %s  
    WHERE activity_id = %s  
    """  
  
    cursor.execute(query, (new_sleep, aid))  
    cnx.commit()  
  
    if cursor.rowcount > 0:  
        print("Record updated")  
    else:  
        print("Record not found")  
  
    cursor.close()
```

```
def delete_activity_data(aid):
```

```

cursor = cnx.cursor()

cursor.execute(
    "DELETE FROM activity_data WHERE activity_id = %s", (aid,)
)
cnx.commit()

if cursor.rowcount > 0:
    print("Record deleted")
else:
    print("Record not found")

cursor.close()

```

4. **Stress_analysis.py**

```

import mysql.connector as ms

cnx = ms.connect(
    host="localhost",
    user="root",
    passwd="1234",
    database="stress_detection"
)

def add_record_stress_analysis():
    cursor = cnx.cursor()
    analyst_id = input("Enter analyst id:")
    user_id = input("Enter user id: ")

```

```
activity_id = input("Enter activity id: ")
```

```
cursor.execute("SELECT sleep_hours, heart_rate, screen_time FROM activity_data \n\n                WHERE activity_id=%s",(activity_id,))
```

```
data = cursor.fetchone()
```

```
sleep, heart, screen = data
```

```
if sleep < 6 or int(heart) > 90 or screen > 7:
```

```
    stress = "High"
```

```
    suggestion = "Sleep more & reduce screen time"
```

```
elif sleep <= 7 or int(heart) >= 75 or screen >= 5:
```

```
    stress = "Medium"
```

```
    suggestion = "Reduce screen time"
```

```
else:
```

```
    stress = "Low"
```

```
    suggestion = "Maintain healthy routine"
```

```
query = ""
```

```
INSERT INTO stress_analyst
```

```
(analyst_id, user_id, activity_id, stress_level, suggestion)
```

```
VALUES (%s, %s, %s, %s, %s)
```

```
""
```

```
cursor.execute(query, (analyst_id, user_id, activity_id, stress, suggestion))
```

```
cnx.commit()
```

```
print("STRESS ANALYSIS RECORD INSERTED")
```

```
cursor.close()
```

```

def display_record_stress_analysis():

    cursor = cnx.cursor()

    query = """
SELECT u.user_id, u.uname,
       a.sleep_hours, a.heart_rate, a.screen_time,
       s.stress_level, s.suggestion
FROM userrecord u
JOIN stress_analyst s ON u.user_id = s.user_id
JOIN activity_data a ON a.activity_id = s.activity_id
"""

    cursor.execute(query)
    records = cursor.fetchall()
    print("REPORT OF STRESS ANALYSIS")

    for row in records:

        print("User ID:", row[0])
        print("Name:", row[1])
        print("Sleep Hours:", row[2])
        print("Heart Rate:", row[3])
        print("Screen Time (IN HOURS) :", row[4])
        print("Stress Level:", row[5])
        print("Suggestion:", row[6])
        print()

```

```
cursor.close()
```

```
def update_record_stress_analysis():
```

```
    cursor=cnx.cursor()
```

```
    activity_id=input("Enter activity id to update stress record:")
```

```
    new_stress=input("Enter new stress level (High/Medium/Low):")
```

```
    new_suggestion=input('Enter new suggestion:')
```

```
    query="""
```

```
    UPDATE stress_analyst
```

```
    SET stress_level=%s,
```

```
    suggestion=%s
```

```
    WHERE activity_id=%s
```

```
    """
```

```
    cursor.execute(query,(new_stress, new_suggestion, activity_id))
```

```
    cnx.commit()
```

```
    if cursor.rowcount>0:
```

```
        print("Stress analysis record updated successfully")
```

```
    else:
```

```
        print("No record found for given activity id:")
```

```
    cursor.close()
```

```
def delete_record_stress_analysis():
```

```
cursor=cnx.cursor()
```

```
activity_id=input('Enter activity id to delete stress record')
```

```
query="DELETE FROM stress_analyst WHERE activity_id=%s"
```

```
cursor.execute(query,(activity_id,))
```

```
cnx.commit()
```

```
if cursor.rowcount>0:
```

```
    print('stress analysis record deleted successfully')
```

```
else:
```

```
    print('no record found for given activity id')
```

```
cursor.close()
```

```
def search_by_activity_id(aid):
```

```
    cursor=cnx.cursor()
```

```
    cursor.execute('SELECT * FROM stress_analyst WHERE activity_id=%s',(aid,))
```

```
    records=cursor.fetchall()
```

```
    print("REPORT OF STRESS ANALYSIS")
```

```
    for row in records:
```

```
        print("AnalystID:", row[0])
```

```
        print("User ID:", row[1])
```

```
        print("Activity ID:", row[2])
```

```
        print("Stress:", row[3])
```

```
        print("Suggestion:",row[4])
```

```
    cursor.close()
```

```

def search_by_user_id(uid):
    cursor=cnx.cursor()
    cursor.execute('SELECT * FROM stress_analyst WHERE user_id=%s',(uid,))
    records=cursor.fetchall()
    print("REPORT OF STRESS ANALYSIS")
    for row in records:
        print("AnalystID:", row[0])
        print("User ID:", row[1])
        print("Activity ID:", row[2])
        print("Stress:", row[3])
        print("Suggestion:",row[4])

    cursor.close()

def search_by_analyst_id(anid):
    cursor=cnx.cursor()
    cursor.execute('SELECT * FROM stress_analyst WHERE analyst_id=%s',(anid,))
    records=cursor.fetchall()
    print("REPORT OF STRESS ANALYSIS")
    for row in records:
        print("AnalystID:", row[0])
        print("User ID:", row[1])
        print("Activity ID:", row[2])
        print("Stress:", row[3])
        print("Suggestion:",row[4])

    cursor.close()

```

5. Menu.py


```
import userrecord
import activityclass
import stress_analysis

def main():
    while True:
        print("Menu")
        print("1. User")
        print("2. Activity")
        print("3. Stress Report")
        print("4. Exit")
        print()
        choice=int(input("Enter choice\t"))
        if choice==1:
            print("User menu")
            print("1. Add User")
            print("2. Display User")
            print("3. Search User")
            print("4. Update User")
            print("5. Delete User")
            print()
            ch=int(input("Enter choice\t"))
            if ch==1:
                userrecord.add_user_data()
            elif ch==2:
                userrecord.display_userrecord_data()
            elif ch==3:
                uid=input('enter user id:')
                stress_analysis.stress_report(uid)
```

```
        userrecord.search_userrecord_by_id(uid)
elif ch==4:
    uname=input('enter username')
    userrecord.update_userrecord(uname)
elif ch==5:
    uid=input('enter user id:')
    userrecord.user_delete(uid)

elif choice==2:
    print("1. Add Activity Data")
    print("2. Display All Activity")
    print("3. Search Activity")
    print("4. Update Activity")
    print("5. Delete Activity")
    print()

ch = int(input("Enter your choice: "))

if ch == 1:
    activityclass.add_activity_data()
elif ch == 2:
    activityclass.display_activity_data()
elif ch == 3:
    aid=input('enter activity id')
    activityclass.search_activity_data(aid)
```

```
elif ch == 4:

    aid=input('Enter activity id')

    activityclass.update_activity_data(aid)

elif ch == 5:

    aid=input('Enter activity id')

    activityclass.delete_activity_data(aid)


elif choice==3:

    print("1. Add Stress Report")

    print("2. Display Stress Report")

    print("3. Update Stress Report")

    print("4. Delete Stress Report")

    print()


    ch=int(input("enter choice"))


    if ch==1:

        stress_analysis.add_record_stress_analysis()

    elif ch==2:

        stress_analysis.display_record_stress_analysis()

    elif ch==3:

        stress_analysis.update_record_stress_analysis()

    elif ch==4:

        stress_analysis.delete_record_stress_analysis()


elif choice==4:

    print('exiting program')
```

break

main()

Output:

```
>>>
===== RESTART: C:\Users\parir\OneDrive\Desktop\PROJECT FOLDER\menu.py =====
Menu
1. User
2. Activity
3. Stress Report
4. Exit

Enter choice    1
User menu
1. Add User
2. Display User
3. Search User
4. Update User
5. Delete User

Enter choice    1
Enter user idU501
Enter user nameRIA
Enter age18
Enter genderFemale
Enter the occupationStudent
RECORD INSERTED

Menu
1. User
2. Activity
3. Stress Report
4. Exit

Enter choice    1
User menu
1. Add User
2. Display User
3. Search User
4. Update User
5. Delete User

Enter choice    2
User id= U501
User name= RIA
User age= 18
User gender= Female
User occupation= Student
Data Displayed

-----
Menu
1. User
2. Activity
3. Stress Report
4. Exit

Enter choice    1
User menu
1. Add User
2. Display User
3. Search User
4. Update User
5. Delete User

Enter choice    4
enter usernameRIA
Enter the new data
Enter user age1000
Enter user genderFF
Enter user occupationSS
Record(s) updated successfully
-----
```

```
Menu
1. User
2. Activity
3. Stress Report
4. Exit
```

```
Enter choice    1
```

```
User menu
```

```
1. Add User
2. Display User
3. Search User
4. Update User
5. Delete User
```

```
Enter choice    2
```

```
User id= U501
```

```
User name= RIA
```

```
User age= 1000
```

```
User gender= FF
```

```
User occupation= SS
```

```
Data Displayed
```

```
Menu
1. User
2. Activity
3. Stress Report
4. Exit
```

```
Enter choice    2
```

```
1. Add Activity Data
2. Display All Activity
3. Search Activity
4. Update Activity
5. Delete Activity
```

```
Enter your choice: 1
```

```
Enter activity id: ACT501
```

```
Enter sleep hours: 7
```

```
Enter heart rate: 82
```

```
Enter screen time: 5
```

```
RECORD INSERTED IN ACTIVITY TABLE
```

```
Menu
1. User
2. Activity
3. Stress Report
4. Exit
```

```
Enter choice    2
```

```
1. Add Activity Data
2. Display All Activity
3. Search Activity
4. Update Activity
5. Delete Activity
```

```
Enter your choice: 2
```

```
Displaying data from activity table
records of the data is as follow
```

```
Activty id= ACT501
```

```
Sleep hours= 7
```

```
Heart rate= 82
```

```
Screen time= 5
```

Menu

1. User
2. Activity
3. Stress Report
4. Exit

Enter choice 2

1. Add Activity Data
2. Display All Activity
3. Search Activity
4. Update Activity
5. Delete Activity

Enter your choice: 3

enter activity idACT501

ACTIVITY DATA REPORT

ActivityID: ACT501

Sleep Hours: 7

Heart Rate: 82

Screen Time: 5

Menu

1. User
2. Activity
3. Stress Report
4. Exit

Enter choice 2

1. Add Activity Data
2. Display All Activity
3. Search Activity
4. Update Activity
5. Delete Activity

Enter your choice: 4

Enter activity idACT501

Enter new sleep hours: 8

Record updated

Menu

1. User
2. Activity
3. Stress Report
4. Exit

Enter choice 2

1. Add Activity Data
2. Display All Activity
3. Search Activity
4. Update Activity
5. Delete Activity

Enter your choice: 2

Displaying data from activity table
records of the data is as follow

Activity id= ACT501

Sleep hours= 8

Heart rate= 82

Screen time= 5

Menu

1. User
2. Activity
3. Stress Report
4. Exit

Enter choice 3

1. Add Stress Report
2. Display Stress Report
3. Update Stress Report
4. Delete Stress Report

enter choicel

Enter analyst id:AN501

Enter user id: U501

Enter activity id: ACT501

STRESS ANALYSIS RECORD INSERTED

--

```
Menu
1. User
2. Activity
3. Stress Report
4. Exit

Enter choice    3
1. Add Stress Report
2. Display Stress Report
3. Update Stress Report
4. Delete Stress Report

enter choice2
REPORT OF STRESS ANALYSIS
User ID: U501
Name: RIA
Sleep Hours: 8
Heart Rate: 82
Screen Time (IN HOURS) : 5
Stress Level: Medium
Suggestion: Reduce screen time
```

```
Menu
1. User
2. Activity
3. Stress Report
4. Exit

Enter choice    3
1. Add Stress Report
2. Display Stress Report
3. Update Stress Report
4. Delete Stress Report

enter choice3
Enter activity id to update stress record:ACT501
Enter new stress level (High/Medium/Low):Low
Enter new suggestion:Maintain healthy routine
Stress analysis record updated successfully
```

```
Menu
1. User
2. Activity
3. Stress Report
4. Exit

Enter choice    3
1. Add Stress Report
2. Display Stress Report
3. Update Stress Report
4. Delete Stress Report

enter choice2
REPORT OF STRESS ANALYSIS
User ID: U501
Name: RIA
Sleep Hours: 8
Heart Rate: 82
Screen Time (IN HOURS) : 5
Stress Level: Low
Suggestion: Maintain healthy routine
```

```
Menu
1. User
2. Activity
3. Stress Report
4. Exit

Enter choice    1
User menu
1. Add User
2. Display User
3. Search User
4. Update User
5. Delete User

Enter choice    5
enter user id:U501
Displaying record for deletion
[('U501', 'RIA', 1000, 'FF', 'SS')]
Do you want to delete the recordy
1 records deleted succesfully
Menu
1. User
2. Activity
3. Stress Report
4. Exit

Enter choice    1
User menu
1. Add User
2. Display User
3. Search User
4. Update User
5. Delete User

Enter choice    2
```

```
Enter choice    2
Menu
1. User
2. Activity
3. Stress Report
4. Exit

Enter choice    2
1. Add Activity Data
2. Display All Activity
3. Search Activity
4. Update Activity
5. Delete Activity

Enter your choice: 5
Enter activity idACT501
Record deleted
Menu
1. User
2. Activity
3. Stress Report
4. Exit

Enter choice    2
1. Add Activity Data
2. Display All Activity
3. Search Activity
4. Update Activity
5. Delete Activity

Enter your choice: 2
```

Menu

1. User
2. Activity
3. Stress Report
4. Exit

Enter choice 3

1. Add Stress Report
2. Display Stress Report
3. Update Stress Report
4. Delete Stress Report

enter choice4

Enter activity id to delete stress recordACT501
stress analysis record deleted successfully

Menu

1. User
2. Activity
3. Stress Report
4. Exit

Enter choice 3

1. Add Stress Report
2. Display Stress Report
3. Update Stress Report
4. Delete Stress Report

enter choice2

REPORT OF STRESS ANANLYSIS

Menu

1. User
2. Activity
3. Stress Report
4. Exit

Enter choice 4

exiting program

>>>

FUTURE SCOPE

The stress detection system can be further enhanced by adding more advanced features. In future, real-time data can be collected using wearable devices such as smartwatches and fitness bands. Machine learning techniques can be used to predict stress levels more accurately.

The system can also be developed as a mobile or web application for easy access by users. Additional parameters like diet, physical exercise and mental health activities can be included to improve stress analysis. This project has wide scope in healthcare and wellness applications.

BIBLIOGRAPHY

Reference materials from python software helpbook